ME315A_FarahLab08

Mateus Farah

Lab 08

```
import polars as pl
basics = pl.read_csv("title.basics0.tsv.gz", separator="\t", null_values=r"\N")
ratings = pl.read_csv("title.ratings.tsv.gz", separator="\t", null_values=r"\N")
principals = pl.read_csv("title.principals0.tsv.gz", separator="\t", null_values=r"\N")
```

1. Quais são os 5 filmes com as maiores notas (averageRating)? Apresente uma solução capaz de desempatar os filmes baseando-se no número de votos recebidos.

```
q1 = (
    basics
    .filter(pl.col("titleType") == "movie")
    .join(ratings, on="tconst", how="inner")
    .sort(by=["averageRating", "numVotes"], descending=[True, True])
    .select(["tconst", "primaryTitle", "averageRating", "numVotes"])
    .head(5)
)

print(q1)
```

```
shape: (5, 4)

 tconst      primaryTitle               averageRating  numVotes
 ---         ---                        ---            ---
 str         str                        f64            i64

 tt33075815  Kaveri                     10.0           1023
 tt27815015  Kurukku                    10.0           451
 tt28450376  Jedal Dar Omghe 30 Metri   10.0           142
```

```
tt27859713   Sargashte                10.0           134
tt33507675   Gorgeous Rascal          10.0           115
```

2. Qual é o gênero mais frequente entre os filmes com nota maior que 8?

```
q2 = (
    basics
    .filter(pl.col("titleType") == "movie")
    .join(ratings, on="tconst", how="inner")
    .filter(pl.col("averageRating") > 8)
    .filter(pl.col("genres").is_not_null())
    .with_columns(pl.col("genres").str.split(","))
    .explode("genres")
    .with_columns(pl.col("genres").str.strip_chars())
    .group_by("genres")
    .len()
    .sort(by="len", descending=True)
    .head(1)
)

print(q2)
```

```
shape: (1, 2)

 genres        len
 ---           ---
 str           u32

 Documentary   10564
```

3.

```
q3 = (
    basics
    .filter(pl.col("titleType") == "movie")
    .join(ratings, on="tconst", how="inner")
    .filter(pl.col("averageRating") > 7.5)
    .join(principals, on="tconst", how="inner")
    .filter(pl.col("category").is_in(["actor", "actress"]))
    .group_by("nconst")
```

```
    .len()
    .sort(by="len", descending=True)
    .head(3)
)

print(q3)
```

```
shape: (3, 2)

 nconst      len
 ---         ---
 str         u32

 nm0004660   231
 nm0595934   155
 nm3183374   124
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-10-14 18:37:39 -03"
```